TECHNICAL ARCHITECTURE

Full System Design Document

School Management Platform · v1.0 Engineering Specification

1. System Overview & High-Level Architecture

This document covers the complete technical design of the school management platform. It is intended as the engineering reference for senior engineers building the system. Every major subsystem is described with data structures, interfaces, algorithms, failure modes, and edge cases.

Scope of This Document

Covers: System topology, Control Plane, Data Plane, Provisioning Pipeline, Schema Migration Engine, Canvas Builder architecture, API layer design, Authentication & RBAC, AI Agent integration, Mobile architecture, Multi-language system, and Monitoring. Does not cover: UI component styling, business logic of individual school modules (fees, attendance, etc.), or third-party integration implementation details.

1.1 Two-Plane Topology

The platform is split into two distinct operational planes that never share data.

Plane	Description
Control Plane	Our platform's global brain. One deployment, owned by us. Manages all school tenants, billing, provisioning, monitoring, and the AI agent router. Tech: Next.js 16 + Fastify v5 + PostgreSQL 16 (single DB). Hosted: GCP Cloud Run + Cloud SQL. Access: only our internal team.
Data Plane (per school)	Each school's fully isolated environment. Separate PostgreSQL instance, separate Redis, separate Cloud Storage bucket. Schema is dynamic — changes per school configuration. Hosted: GCP Cloud SQL per-school instance or cluster. Access: only via the platform API, never directly.

Diagram — Two-Plane Topology

[Browser / Mobile App] → [Next.js 16 Frontend (Cloud Run)] → [Fastify API Gateway (Cloud Run)] → [Auth Middleware] → [Tenant Router] → splits to → [Control Plane DB (single PG)] for platform

ops, OR → [School Data Plane (per-school PG + Redis)] for school data. The Provisioning Service lives in Control Plane and provisions Data Plane resources. The AI Agent router lives in Control Plane and proxies to Claude API with per-school context.

1.2 Request Routing Model

Every API request carries a tenant identifier resolved from the authenticated session. The tenant router maps this to the correct school's database connection string, which is fetched from GCP Secret Manager and cached in the API process memory (TTL: 5 minutes).

```
// Simplified tenant routing middleware (Fastify)
fastify.addHook('preHandler', async (req, reply) => {
  const session = await verifyJWT(req.headers.authorization);
  const school  = await controlPlaneDB.schools.findUnique({
    where: { id: session.schoolId },
    select: { id:true, dbSecretId:true, redisSecretId:true, status:true }
  });
  if (!school || school.status !== 'ACTIVE') return reply.code(403).send();
  req.tenantCtx = {
    schoolId:    school.id,
    db:          await getPooledConnection(school.dbSecretId),   // cached
    redis:       await getRedisClient(school.redisSecretId),     // cached
    permissions: session.permissions,
    role:        session.role,
  };
});
```

1.3 Core Data Flow

Below is the end-to-end data flow for a typical school user action, covering all layers from browser to database.

Layer	Trigger	What Happens
1. Browser	User action (click, form submit, canvas drag)	React 19.2 component fires state change or API call via fetch/WebSocket
2. Next.js API Route / Fastify	HTTP request arrives at API gateway	JWT verified, tenant context resolved, RBAC permission checked
3. Schema Engine (if schema change)	Canvas change detected	Schema Migration Engine parses change descriptor, validates, executes migration
4. Business Logic Layer	Standard CRUD or domain logic	Fastify route handler executes against school's PostgreSQL via Prisma or raw SQL
5. PostgreSQL (school DB)	Query executed	Data stored, updated, or returned. All within isolated school database.
6. Redis (school)	Side effects fired	Cache invalidated, pub/sub event published, BullMQ job queued if needed
7. Response	API returns response	Next.js renders updated UI. WebSocket

		broadcasts to other connected users if relevant.

2. Control Plane Design

The Control Plane is the platform's central nervous system. It is a single, highly available deployment managed exclusively by us. It never stores school academic or financial data — only operational metadata about schools and the platform itself.

2.1 Control Plane Database Schema

PostgreSQL 16. The following are the core tables in the Control Plane database.

```
-- schools: one row per subscribed school
CREATE TABLE schools (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name            TEXT NOT NULL,
  slug            TEXT UNIQUE NOT NULL,          -- url-safe identifier
  tier            TEXT NOT NULL,                 --
'starter'|'growth'|'enterprise'
  status          TEXT NOT NULL DEFAULT 'PROVISIONING',
                                                 -- PROVISIONING|ACTIVE|
SUSPENDED|CANCELLED
  region          TEXT NOT NULL DEFAULT 'asia-south1',
  db_secret_id    TEXT,                          -- GCP Secret Manager key for
DB creds
  redis_secret_id TEXT,                          -- GCP Secret Manager key for
Redis creds
  storage_bucket  TEXT,                          -- GCP Cloud Storage bucket
name
  db_instance_id  TEXT,                          -- GCP Cloud SQL instance name
  provisioned_at  TIMESTAMPTZ,
  cancelled_at    TIMESTAMPTZ,
  created_at      TIMESTAMPTZ DEFAULT now(),
  updated_at      TIMESTAMPTZ DEFAULT now()
);

-- provision_log: audit trail of every provisioning event
CREATE TABLE provision_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id   UUID REFERENCES schools(id),
  event       TEXT NOT NULL,   -- STARTED|DB_READY|REDIS_READY|SEEDED|
ADMIN_CREATED|COMPLETE|FAILED
  payload     JSONB,           -- event-specific metadata
  error       TEXT,            -- null unless FAILED
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- subscriptions: billing records
CREATE TABLE subscriptions (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id       UUID REFERENCES schools(id),
  tier            TEXT NOT NULL,
  billing_cycle   TEXT NOT NULL,         -- 'monthly'|'annual'
  amount_paise    BIGINT NOT NULL,       -- stored in paise (1 INR = 100 paise)
  status          TEXT NOT NULL,         -- 'active'|'past_due'|'cancelled'
```

```
  current_period_start  TIMESTAMPTZ,
  current_period_end    TIMESTAMPTZ,
  razorpay_sub_id       TEXT,            -- Razorpay subscription ID
  created_at      TIMESTAMPTZ DEFAULT now()
);
```

2.2 TypeScript Interfaces — Control Plane

```
// types/control-plane.ts

export type SchoolStatus = 'PROVISIONING' | 'ACTIVE' | 'SUSPENDED' |
'CANCELLED';
export type Tier         = 'starter' | 'growth' | 'enterprise';
export type BillingCycle = 'monthly' | 'annual';

export interface School {
  id:             string;
  name:           string;
  slug:           string;
  tier:           Tier;
  status:         SchoolStatus;
  region:         string;
  dbSecretId:     string | null;
  redisSecretId:  string | null;
  storageBucket:  string | null;
  dbInstanceId:   string | null;
  provisionedAt:  Date | null;
  cancelledAt:    Date | null;
  createdAt:      Date;
  updatedAt:      Date;
}

export interface TenantContext {
  schoolId:    string;
  db:          PostgresPool;
  redis:       RedisClient;
  permissions: PermissionSet;
  role:        string;
  userId:      string;
}
```

3. Provisioning Pipeline — Full Design

When a school completes payment, the Provisioning Service runs a fully automated, idempotent, step-by-step pipeline to create the school's isolated cloud environment. The pipeline is designed to be resumable — if it fails at any step, it can be retried from that step without re-running completed steps.

3.1 Pipeline State Machine

The pipeline is modelled as a state machine. Each step transitions the school record from one state to the next. On failure, the school is moved to PROVISION_FAILED and an alert is fired to our ops team.

```
type ProvisionState =
  | 'PENDING'
  | 'DB_PROVISIONING'
  | 'DB_READY'
  | 'REDIS_PROVISIONING'
  | 'REDIS_READY'
  | 'STORAGE_PROVISIONING'
  | 'STORAGE_READY'
  | 'SCHEMA_SEEDING'
  | 'SCHEMA_SEEDED'
  | 'ADMIN_CREATING'
  | 'ACTIVE'
  | 'PROVISION_FAILED';

// State transitions:
// PENDING → DB_PROVISIONING → DB_READY
//         → REDIS_PROVISIONING → REDIS_READY
//         → STORAGE_PROVISIONING → STORAGE_READY
//         → SCHEMA_SEEDING → SCHEMA_SEEDED
//         → ADMIN_CREATING → ACTIVE
// Any step → PROVISION_FAILED (with error logged)
```

3.2 Provisioning Service — Algorithm

```
// services/provisioning/provision.ts

export async function provisionSchool(schoolId: string): Promise<void> {
  const school = await db.schools.findUnique({ where: { id: schoolId } });
  if (!school) throw new Error('School not found');

  // Idempotency: resume from last successful state
  const startFrom = school.status as ProvisionState;
  const steps     = getPipelineSteps(startFrom);

  for (const step of steps) {
    await logProvisionEvent(schoolId, step.name, 'STARTED');
    try {
      await step.execute(school);
      await db.schools.update({
```

```
        where: { id: schoolId },
        data:  { status: step.successState, updatedAt: new Date() }
      });
      await logProvisionEvent(schoolId, step.name, 'COMPLETE');
    } catch (err) {
      await db.schools.update({
        where: { id: schoolId },
        data:  { status: 'PROVISION_FAILED' }
      });
      await logProvisionEvent(schoolId, step.name, 'FAILED', { error:
String(err) });
      await alertOpsTeam({ schoolId, step: step.name, error: String(err) });
      throw err;  // stop the pipeline
    }
  }
}

// Step 1: Create Cloud SQL PostgreSQL instance
async function provisionDatabase(school: School): Promise<void> {
  const instanceName = `school-${school.slug}-${Date.now()}`;
  await gcpSQLClient.instances.create({
    project:  GCP_PROJECT_ID,
    resource: {
      name:            instanceName,
      databaseVersion: 'POSTGRES_16',
      region:          school.region,
      settings: {
        tier:              getTierMachineType(school.tier), // db-g1-small /
db-n1-standard-2 etc
        backupConfiguration: { enabled: true, startTime: '02:00' },
        ipConfiguration:    { ipv4Enabled: false, privateNetwork: VPC_NETWORK },
      }
    }
  });
  // Wait for instance to be RUNNABLE (poll with backoff)
  await waitForSQLInstance(instanceName);
  // Generate strong random password
  const dbPassword = crypto.randomBytes(32).toString('base64url');
  // Store credentials in Secret Manager — never in DB or env vars
  const secretId = await storeSecret(`school-${school.slug}-db`, {
    host:     getInstancePrivateIP(instanceName),
    port:     5432,
    database: 'schooldb',
    username: 'app_user',
    password: dbPassword,
  });
  await db.schools.update({
    where: { id: school.id },
    data:  { dbSecretId: secretId, dbInstanceId: instanceName }
  });
}
```

3.3 Core Schema Seeder

Once the database is ready, the Schema Seeder runs all base migrations to create the default school tables. These are the tables every school starts with before any customization.

```
// The seeder runs a versioned SQL migration set against the new DB
async function seedCoreSchema(schoolConn: PostgresPool): Promise<void> {
  const migrations = await loadMigrations('./migrations/core/');
  // migrations are sorted by version number: 001_init.sql, 002_students.sql ...
  for (const migration of migrations) {
    await schoolConn.query('BEGIN');
    try {
      await schoolConn.query(migration.sql);
      await schoolConn.query(
        'INSERT INTO _migration_history (version, name, applied_at) VALUES
($1,$2,now())',
        [migration.version, migration.name]
      );
      await schoolConn.query('COMMIT');
    } catch (err) {
      await schoolConn.query('ROLLBACK');
      throw err;
    }
  }
}
```

The core schema includes the following default tables across all school tiers:

Table	Purpose
_migration_history	Tracks all schema migrations ever applied to this school's DB.
_meta_schema	The platform's internal JSON record of this school's current data model — what tables exist, what columns, what types. This is the Schema Engine's source of truth.
users	All users (staff, students, parents) with role references.
roles	Dynamic role definitions created via the canvas builder.
role_permissions	Junction table: role → permission mapping. Generated by the Role Builder canvas.
students	Core student profile fields. Extended dynamically per school.
staff	Staff/employee records. Extended dynamically.
classes	Academic classes/sections.
attendance	Daily and period-wise attendance records.
fee_structures	Fee templates by class/category/installment.
fee_collections	Individual fee payment records.
timetable_slots	Period-to-subject-to-teacher mappings.
branches	School branch definitions for multi-branch schools.

workflow_definitions	Serialized workflow JSON created via the Workflow Builder canvas.
workflow_instances	Running instances of workflows (e.g., an active admission process).
notifications	Notification log for all system-generated alerts.

3.4 Failure Scenarios — Provisioning

Critical: Partial Provisioning

If the process crashes after DB is created but before secrets are stored, the DB instance exists in GCP but the platform has no record of its credentials. The ops alerting system must detect PROVISION_FAILED states and trigger cleanup: delete the orphaned Cloud SQL instance, then retry from scratch. Never leave orphaned resources.

Failure	Trigger	Recovery Strategy
DB create times out	GCP Cloud SQL creation exceeds 15-minute threshold	Cancel request, delete any partial instance, set status=PROVISION_FAILED, alert ops team, auto-retry once after 10 minutes
Secret Manager write fails	GCP Secret Manager unavailable or quota exceeded	Retry 3x with exponential backoff (1s, 2s, 4s). If all fail, mark PROVISION_FAILED. DB instance remains — retry picks up from DB_READY state.
Schema seeder SQL error	A core migration SQL fails (syntax, constraint)	ROLLBACK transaction. Mark SCHEMA_SEEDING_FAILED. Alert ops. Fix migration, redeploy seeder, retry from SCHEMA_SEEDING state.
Admin email delivery fails	Welcome email not delivered	School is ACTIVE — this is not a blocking failure. Queue email retry via BullMQ. Show 'resend email' button in our ops dashboard.
Concurrent duplicate trigger	Payment webhook fires twice for same school	Idempotency key on provisioning job prevents double-provisioning. If school.status != PENDING, skip silently and log.

4. Schema Migration Engine — Full Design

The Schema Migration Engine (SME) is the most technically complex component in the platform. It enables real-time, synchronous, user-driven changes to the database schema of any school — triggered by UI actions on the canvas builder — without data loss, without downtime, and with full rollback capability.

> **Core Invariants**
> (1) A schema change either fully succeeds or fully rolls back — no partial migrations. (2) No existing data is ever deleted without explicit admin confirmation. (3) Every migration is logged as an immutable event — the full history can be replayed. (4) The meta-schema is always consistent with the actual database schema.

4.1 Meta-Schema Registry

The Meta-Schema Registry is the heart of the SME. It is a JSON document stored in the school's own database (table: _meta_schema) that describes the current logical data model of the school. Every field, every table, every relationship is recorded here. When the frontend needs to know what fields exist on the 'students' table, it reads the meta-schema — it never introspects the database directly.

```
// types/meta-schema.ts

export type FieldType =
  | 'TEXT' | 'INTEGER' | 'DECIMAL' | 'BOOLEAN'
  | 'DATE' | 'TIMESTAMPTZ' | 'UUID'
  | 'ENUM' | 'JSONB' | 'FILE_REF';

export interface MetaField {
  id:           string;          // stable UUID, never changes even if column
renamed
  columnName:   string;          // actual PostgreSQL column name
  displayName:  string;          // label shown in UI
  type:         FieldType;
  nullable:     boolean;
  defaultValue: unknown | null;
  enumValues:   string[] | null; // only if type === 'ENUM'
  isCore:       boolean;         // true = seeded by platform, cannot be deleted
  isHidden:     boolean;         // soft-deleted fields: hidden but not dropped
  createdAt:    string;          // ISO timestamp
  createdBy:    string;          // userId who added this field
}

export interface MetaTable {
  id:           string;
  tableName:    string;          // actual PostgreSQL table name
  displayName:  string;          // label in UI (e.g., 'Students')
  isCore:       boolean;
  isHidden:     boolean;
  fields:       MetaField[];
  createdAt:    string;
```

```
}

export interface MetaSchema {
  version:     number;          // increments on every change
  schoolId:    string;
  tables:      MetaTable[];
  updatedAt:   string;
}
```

4.2 Change Descriptor — What the Canvas Sends

When a user saves a canvas change, the frontend constructs a typed Change Descriptor and sends it to the SME endpoint. The SME never receives raw SQL — it receives a structured description of the intent.

```
// types/schema-change.ts

export type ChangeAction =
  | 'ADD_TABLE'
  | 'DROP_TABLE'          // soft-delete only unless confirmed
  | 'ADD_COLUMN'
  | 'DROP_COLUMN'         // soft-delete only unless confirmed
  | 'RENAME_COLUMN'
  | 'CHANGE_COLUMN_TYPE'  // only allowed when no data exists in column
  | 'ADD_ENUM_VALUE'
  | 'SET_DEFAULT'
  | 'SET_NULLABLE';

export interface SchemaChangeDescriptor {
  action:      ChangeAction;
  tableId:     string;          // MetaTable.id
  fieldId?:    string;          // MetaField.id — for column-level actions
  payload:     AddTablePayload
             | AddColumnPayload
             | DropColumnPayload
             | RenameColumnPayload
             | ChangeTypePayload;
  requestedBy: string;          // userId
  timestamp:   string;
}

export interface AddColumnPayload {
  columnName:   string;
  displayName:  string;
  type:         FieldType;
  nullable:     boolean;
  defaultValue: unknown | null;
  enumValues?:  string[];
}

export interface AddTablePayload {
  tableName:   string;
```

```
  displayName: string;
  fields:      AddColumnPayload[];  // initial fields for the new table
}
```

4.3 SME Processing Algorithm

The full algorithm executed by the Schema Migration Engine for every incoming change descriptor.

```
// services/schema-engine/engine.ts

export async function applySchemaChange(
  ctx:        TenantContext,
  descriptor: SchemaChangeDescriptor
): Promise<SchemaChangeResult> {

  // ── STEP 1: LOAD CURRENT META-SCHEMA ──────────────────────────
  const metaSchema = await loadMetaSchema(ctx.db);

  // ── STEP 2: VALIDATE DESCRIPTOR ───────────────────────────────
  const validation = validateDescriptor(descriptor, metaSchema);
  if (!validation.ok) {
    return { success: false, error: validation.error };
  }

  // ── STEP 3: GENERATE MIGRATION SQL ────────────────────────────
  const migration = generateMigrationSQL(descriptor, metaSchema);
  // migration = { forwardSQL: string[], rollbackSQL: string[] }

  // ── STEP 4: COMPUTE NEW META-SCHEMA ───────────────────────────
  const newMetaSchema = applyDescriptorToMeta(descriptor, metaSchema);

  // ── STEP 5: EXECUTE — TRANSACTIONAL ───────────────────────────
  await ctx.db.query('BEGIN');
  try {
    // 5a. Run forward SQL
    for (const sql of migration.forwardSQL) {
      await ctx.db.query(sql);
    }
    // 5b. Write new meta-schema
    await saveMetaSchema(ctx.db, newMetaSchema);
    // 5c. Write migration event log
    await writeMigrationLog(ctx.db, {
      version:     newMetaSchema.version,
      descriptor,
      forwardSQL:  migration.forwardSQL,
      rollbackSQL: migration.rollbackSQL,
      appliedBy:   descriptor.requestedBy,
      appliedAt:   new Date().toISOString(),
    });
    await ctx.db.query('COMMIT');
    return { success: true, newVersion: newMetaSchema.version };
```

```
  } catch (err) {
    await ctx.db.query('ROLLBACK');
    return { success: false, error: String(err) };
  }
}
```

4.4 Validation Rules

Before any SQL is generated, the descriptor is validated against these rules. Violations are returned as user-friendly error messages to the canvas UI.

Rule	Condition	Response
Reserved name check	Column/table name matches a PostgreSQL reserved word or platform internal name	Return error: 'This name is reserved. Choose a different name.'
Duplicate name check	Column with same name already exists in target table	Return error: 'A field with this name already exists.'
Type change with data	CHANGE_COLUMN_TYPE requested on column with existing rows	Return error: 'Cannot change type of a field that has data. Archive existing data first.'
Core table drop	DROP_TABLE on a table with isCore: true	Return error: 'Platform core tables cannot be deleted.'
Core field drop	DROP_COLUMN on a field with isCore: true	Return error: 'This field is required by the platform and cannot be removed.'
Circular reference	ADD_TABLE with a foreign key that creates a circular dependency	Return error: 'This would create a circular relationship. Use a junction table instead.'
Name length	Column or table name exceeds 63 characters (PostgreSQL limit)	Return error: 'Name must be 63 characters or fewer.'
SQL injection check	Any name field contains characters outside [a-z0-9_]	Return error: 'Names can only contain lowercase letters, numbers, and underscores.'

4.5 Migration SQL Generation

The SQL generator converts a validated Change Descriptor into forward and rollback SQL arrays. All SQL is parameterised where possible; table and column names are sanitized through a whitelist check before interpolation.

```
function generateMigrationSQL(
  d:    SchemaChangeDescriptor,
  meta: MetaSchema
): { forwardSQL: string[]; rollbackSQL: string[] } {

  const table = meta.tables.find(t => t.id === d.tableId)!;

  switch (d.action) {
```

```
    case 'ADD_COLUMN': {
      const p = d.payload as AddColumnPayload;
      const pgType = mapToPgType(p.type, p.enumValues);
      const nullStr = p.nullable ? '' : ' NOT NULL';
      const defStr  = p.defaultValue != null
        ? ` DEFAULT ${escapeLiteral(p.defaultValue)}` : '';
      return {
        forwardSQL:  [`ALTER TABLE ${table.tableName}`,
                      `  ADD COLUMN ${p.columnName} ${pgType}${nullStr}$
{defStr};`],
        rollbackSQL: [`ALTER TABLE ${table.tableName}`,
                      `  DROP COLUMN IF EXISTS ${p.columnName};`],
      };
    }

    case 'ADD_TABLE': {
      const p = d.payload as AddTablePayload;
      const cols = p.fields.map(f => {
        const pgType = mapToPgType(f.type, f.enumValues);
        const nullStr = f.nullable ? '' : ' NOT NULL';
        return `  ${f.columnName} ${pgType}${nullStr}`;
      });
      return {
        forwardSQL: [
          `CREATE TABLE ${p.tableName} (`,
          `  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),`,
          `  school_id UUID NOT NULL,`,
          cols.join(',\n') + ',',
          `  created_at TIMESTAMPTZ DEFAULT now(),`,
          `  updated_at TIMESTAMPTZ DEFAULT now()`,
          `);`,
          `CREATE INDEX ON ${p.tableName} (school_id);`,
        ],
        rollbackSQL: [`DROP TABLE IF EXISTS ${p.tableName};`],
      };
    }

    case 'DROP_COLUMN': {
      // Soft delete: hide in meta-schema, never drop the actual column
      // Hard drop only via explicit admin confirmation flow
      return {
        forwardSQL:  [],   // no SQL — only meta-schema marks field
isHidden=true
        rollbackSQL: [],
      };
    }

    // ... other cases
  }
}
```

4.6 Migration Event Log

Every schema change is written as an immutable event to the _migration_log table in the school's database. This serves as the complete audit trail and enables point-in-time schema reconstruction.

```
CREATE TABLE _migration_log (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  version      INTEGER NOT NULL,       -- incremental version of meta-schema
  action       TEXT NOT NULL,          -- the ChangeAction enum value
  descriptor   JSONB NOT NULL,         -- full SchemaChangeDescriptor
  forward_sql  TEXT[] NOT NULL,        -- SQL that was executed
  rollback_sql TEXT[] NOT NULL,        -- SQL to undo this change
  applied_by   UUID NOT NULL,          -- userId
  applied_at   TIMESTAMPTZ DEFAULT now()
);

-- Rollback algorithm: to roll back to version N:
-- 1. SELECT rollback_sql FROM _migration_log WHERE version > N ORDER BY version
DESC
-- 2. Execute each rollback_sql array in order (newest first)
-- 3. Restore _meta_schema.data to the snapshot at version N
-- 4. Delete _migration_log rows WHERE version > N
```

4.7 Edge Cases & Failure Modes — Schema Engine

Data Loss Prevention

DROP_COLUMN never issues DROP COLUMN SQL unless the admin explicitly confirms after seeing a data impact report showing how many rows have non-null values in that column. The UI blocks the operation and shows: 'This field has data in N records. Deleting it will permanently erase this data. Type DELETE to confirm.'

Edge Case	Scenario	Handling
Concurrent edits	Two admins edit canvas simultaneously	Optimistic locking on meta-schema version. Second write fails with 'Schema was modified by another user. Refresh and retry.' Frontend shows merge conflict UI.
Migration times out	ALTER TABLE on a large table (100k+ rows) exceeds 30s	Use non-blocking DDL where possible (ADD COLUMN with DEFAULT is instant in PG 16 via fast-path). For unavoidable locks, queue migration as background job and show progress bar to admin.
Rollback SQL fails	An error occurs while trying to rollback a failed migration	This is a critical state. Log to Sentry with CRITICAL severity. Alert ops team immediately. Do not attempt further automatic rollback. Ops team manually resolves using the migration log.
Meta-schema out of sync	DB schema differs from meta-schema record (e.g., manual intervention)	Nightly consistency checker compares actual pg_catalog against meta-schema. Discrepancies logged and alerted. Manual resolution by ops.

Type coercion failure	Existing data cannot be cast to new column type	Blocked at validation step — CHANGE_COLUMN_TYPE requires zero rows in column. If rows exist, user must archive or clear data first.
Schema version conflict	Rollback attempted on version that other changes depend on	Dependency graph maintained in meta-schema. Rolling back version N is blocked if any field in version N+1 or later references a field added in version N.

5. Canvas Builder Architecture

The Canvas Builder is the frontend system that powers both the Role Builder and Workflow Builder. It is built on React Flow as the graph engine (completely reskinned), Zustand for state management, and Motion for animations. The canvas communicates schema changes to the SME via a typed event queue.

5.1 Canvas State Model

```
// types/canvas.ts

export type NodeType = 'ROLE' | 'WORKFLOW_TRIGGER' | 'WORKFLOW_ACTION'
                     | 'WORKFLOW_CONDITION' | 'WORKFLOW_END';

export interface CanvasNode {
  id:       string;
  type:     NodeType;
  position: { x: number; y: number };
  data:     RoleNodeData | WorkflowNodeData;
  selected: boolean;
}

export interface CanvasEdge {
  id:        string;
  source:    string;   // node id
  target:    string;   // node id
  label?:    string;   // condition label for workflow branches
  animated:  boolean;
}

export interface RoleNodeData {
  roleId:      string;
  roleName:    string;
  permissions: PermissionSet;
  userCount:   number;    // how many users are currently in this role
  isCore:      boolean;
}

export interface WorkflowNodeData {
  workflowId:   string;
  label:        string;
  triggerType?: 'FORM_SUBMIT' | 'DATE_EVENT' | 'PAYMENT' | 'MANUAL';
  actionType?:  'NOTIFY' | 'CREATE_RECORD' | 'ASSIGN_TASK' | 'REQUIRE_APPROVAL'
| 'GENERATE_DOC';
  config:       Record<string, unknown>;  // action-specific config
}

export interface CanvasState {
  nodes:       CanvasNode[];
  edges:       CanvasEdge[];
  selectedIds: Set<string>;
  history:     CanvasSnapshot[];   // undo stack
  future:      CanvasSnapshot[];   // redo stack
```

```
  isDirty:     boolean;
  isSaving:    boolean;
  lastSaved:   Date | null;
}
```

5.2 Zustand Store — Canvas

```
// stores/canvas.store.ts
import { create } from 'zustand';
import { immer } from 'zustand/middleware/immer';

export const useCanvasStore = create<CanvasStore>()(immer((set, get) => ({
  ...initialState,

  addNode: (node: CanvasNode) => set(state => {
    state.history.push(snapshot(state));
    state.future = [];
    state.nodes.push(node);
    state.isDirty = true;
  }),

  moveNode: (id: string, position: {x:number, y:number}) => set(state => {
    const node = state.nodes.find(n => n.id === id);
    if (node) { node.position = position; state.isDirty = true; }
  }),

  connectNodes: (edge: CanvasEdge) => set(state => {
    state.history.push(snapshot(state));
    state.future = [];
    state.edges.push(edge);
    state.isDirty = true;
  }),

  undo: () => set(state => {
    if (!state.history.length) return;
    state.future.push(snapshot(state));
    const prev = state.history.pop()!;
    state.nodes = prev.nodes;
    state.edges = prev.edges;
  }),

  redo: () => set(state => {
    if (!state.future.length) return;
    state.history.push(snapshot(state));
    const next = state.future.pop()!;
    state.nodes = next.nodes;
    state.edges = next.edges;
  }),

  save: async () => {
    const { nodes, edges } = get();
    set(s => { s.isSaving = true; });
```

```
    const descriptors = diffCanvasState(get().lastSavedSnapshot, { nodes,
edges });
    for (const descriptor of descriptors) {
      await schemaEngineClient.applyChange(descriptor);
    }
    set(s => { s.isSaving = false; s.isDirty = false; s.lastSaved = new
Date(); });
  },
})));
```

5.3 Canvas Diff Algorithm

When the user saves the canvas, the system diffs the current state against the last saved snapshot to compute what changed. Only actual changes are sent to the SME as descriptors — not the entire canvas state.

```
function diffCanvasState(
  prev: CanvasSnapshot,
  curr: CanvasSnapshot
): SchemaChangeDescriptor[] {
  const descriptors: SchemaChangeDescriptor[] = [];

  // Added nodes → ADD_TABLE or new role
  const addedNodes = curr.nodes.filter(n => !prev.nodes.find(p => p.id ===
n.id));
  for (const node of addedNodes) {
    if (node.type === 'ROLE') {
      descriptors.push(buildRoleCreateDescriptor(node));
    }
    // Workflow nodes don't generate schema changes — they update
workflow_definitions JSON
  }

  // Removed nodes → DROP (soft)
  const removedNodes = prev.nodes.filter(n => !curr.nodes.find(c => c.id ===
n.id));
  for (const node of removedNodes) {
    descriptors.push(buildDropDescriptor(node));
  }

  // Modified nodes → permission changes, field additions
  const modifiedNodes = curr.nodes.filter(n => {
    const prevNode = prev.nodes.find(p => p.id === n.id);
    return prevNode && !deepEqual(prevNode.data, n.data);
  });
  for (const node of modifiedNodes) {
    descriptors.push(...buildModifyDescriptors(node, prev.nodes.find(p => p.id
=== node.id)!));
  }

  return descriptors;
}
```

5.4 Role Permission System

Permissions are defined at four granularity levels. The Role Builder canvas generates a PermissionSet which is stored in the role_permissions table and cached in Redis for fast API-level enforcement.

```
// types/permissions.ts

export type ResourceType = string;   // module names: 'students', 'fees',
'payroll' etc
export type ActionType   = 'CREATE' | 'READ' | 'UPDATE' | 'DELETE' | 'EXPORT' |
'APPROVE';

export interface FieldPermission {
  fieldId:  string;
  readable: boolean;
  writable: boolean;
}

export interface ResourcePermission {
  resource:         ResourceType;
  actions:          ActionType[];
  fieldPermissions: FieldPermission[];   // field-level control
  rowFilter?:       string;              // e.g., 'branch_id = :userBranchId'
}

export interface PermissionSet {
  roleId:      string;
  permissions: ResourcePermission[];
  // Enforced at API level in every Fastify route handler
}

// API enforcement — called in every route handler
export function assertPermission(
  ctx:       TenantContext,
  resource:  ResourceType,
  action:    ActionType
): void {
  const perm = ctx.permissions.permissions.find(p => p.resource === resource);
  if (!perm || !perm.actions.includes(action)) {
    throw new ForbiddenError(`Action ${action} not permitted on ${resource}`);
  }
}
```

Canvas Edge Case — Role Deletion With Active Users

When a role node is deleted from the canvas, the system checks if any users are currently assigned to that role. If yes, deletion is blocked and the canvas shows: 'X users are assigned to this role. Reassign them before deleting.' The Role Builder shows a reassignment UI where the admin drags users to a new role before the deletion is permitted.

6. API Layer Design

The API is built on Fastify v5 with TypeScript. It serves as the single gateway between all frontend clients (web, mobile) and the data plane. Every route is tenant-scoped, permission-checked, and rate-limited.

6.1 Route Structure

```
// Route namespacing convention
//
// /api/v1/control/*        — Control Plane routes (platform admin only)
// /api/v1/platform/*       — Provisioning, billing, subscription (our ops team)
// /api/v1/school/*         — All school data routes (tenant-scoped)
// /api/v1/schema/*         — Schema Migration Engine endpoints
// /api/v1/canvas/*         — Canvas save/load endpoints
// /api/v1/ai/*             — AI agent proxy endpoints
// /api/v1/auth/*           — Authentication (login, refresh, logout)
// /ws/*                    — WebSocket upgrade endpoints

// Example school route (Fastify plugin pattern)
export async function studentsRoutes(fastify: FastifyInstance) {
  fastify.get('/api/v1/school/students', {
    preHandler: [authenticate, checkPermission('students', 'READ')],
    schema: { querystring: ListStudentsQuerySchema, response: { 200:
StudentsResponseSchema } }
  }, async (req, reply) => {
    const { db, schoolId } = req.tenantCtx;
    const metaSchema = await loadMetaSchema(db);
    // Build dynamic SELECT based on fields the requesting role can read
    const allowedFields = getReadableFields(req.tenantCtx.permissions,
'students', metaSchema);
    const rows = await db.query(
      `SELECT ${allowedFields.join(',')} FROM students WHERE school_id = $1`,
      [schoolId]
    );
    return reply.send({ data: rows, meta: { total: rows.length } });
  });
}
```

6.2 Rate Limiting

Endpoint Type	Rate Limit
Standard routes	100 req/min per user, 1000 req/min per school
Schema change endpoint	10 req/min per user — schema migrations are expensive
AI agent endpoint	30 req/min per user — Claude API has its own limits
Auth endpoints	10 req/min per IP — brute-force protection
File upload	5 req/min per user

Report generation	5 req/min per user — async queued, not synchronous

6.3 Real-Time — WebSocket Design

WebSockets are used for live dashboard updates, attendance broadcasting, canvas collaboration, and notification push. Each school has its own Socket.io namespace, isolated from other schools.

```
// WebSocket namespace per school: /ws/school-{schoolId}
// Rooms within namespace:
//   dashboard        — live KPI updates
//   attendance-{classId} — real-time attendance marking
//   canvas-{canvasId}   — multi-user canvas editing presence
//   notifications   — per-user notification push

io.of(/^\/ws\/school-(.+)$/).use(async (socket, next) => {
  const schoolId = socket.nsp.name.replace('/ws/school-', '');
  const token    = socket.handshake.auth.token;
  const session  = await verifyJWT(token);
  if (session.schoolId !== schoolId) return next(new Error('Unauthorized'));
  socket.data.tenantCtx = await buildTenantContext(session);
  next();
});
```

7. Authentication & Role-Based Access Control

7.1 Auth Flow

Authentication uses JWTs with short-lived access tokens (15 minutes) and long-lived refresh tokens (30 days, stored in httpOnly cookies). The access token carries the user's role and a compact permission fingerprint. Full permissions are loaded from Redis cache on first use.

```
// JWT payload structure
interface JWTPayload {
  sub:         string;   // userId
  schoolId:    string;
  role:        string;
  permHash:    string;   // SHA-256 of permission set — used to detect stale
cache
  iat:         number;
  exp:         number;   // 15 minutes
}

// Permission cache strategy
// Key: perm:{schoolId}:{userId}
// TTL: 5 minutes
// Invalidated on: role change, permission change, password reset
async function getPermissions(ctx: TenantContext): Promise<PermissionSet> {
  const cacheKey = `perm:${ctx.schoolId}:${ctx.userId}`;
  const cached   = await ctx.redis.get(cacheKey);
  if (cached) return JSON.parse(cached);
  const perms = await ctx.db.query(
    `SELECT rp.* FROM role_permissions rp
     JOIN users u ON u.role_id = rp.role_id
     WHERE u.id = $1`, [ctx.userId]
  );
  await ctx.redis.setex(cacheKey, 300, JSON.stringify(perms));
  return perms;
}
```

7.2 Super Admin Account

The super admin is a special user created during provisioning. It has a hardcoded SUPER_ADMIN role that bypasses all permission checks and has access to every resource, every module, and the canvas builder itself. There is exactly one super admin per school. The super admin can create other admin users with limited-scope admin permissions.

> **Super Admin Security**
> Super admin credentials are never stored in the school database. They are issued via the provisioning pipeline and linked to the Control Plane user table. If a super admin loses access, only our ops team can issue a reset via a secured internal admin panel — not via the standard password reset flow.

8. AI Agent — Architecture & Context Design

The AI agent is a context-aware, school-specific assistant powered by the Claude API. It has two interaction modes: chat (sidebar, available on every screen) and voice (STT via Sarvam AI, TTS via Sarvam AI). The agent can read and write to the school's data plane with the same permission constraints as any other user.

8.1 Context Assembly

Every AI agent request assembles a context package before calling the Claude API. This package contains: the school's meta-schema (so the agent knows what data exists), the requesting user's role and permissions, the current screen/module context, and the last 10 turns of the conversation.

```
// services/ai/context.ts

export async function buildAgentContext(
  ctx:        TenantContext,
  screenCtx:  ScreenContext,
  history:    ConversationTurn[]
): Promise<string> {

  const metaSchema  = await loadMetaSchema(ctx.db);
  const schoolInfo  = await getSchoolProfile(ctx.db);
  const userInfo    = await getUserProfile(ctx.db, ctx.userId);

  return `
You are the AI assistant for ${schoolInfo.name}, a school management platform.
You are speaking with ${userInfo.name} who has the role: ${ctx.role}.

CURRENT SCREEN: ${screenCtx.module} > ${screenCtx.page}

SCHOOL DATA MODEL (tables and fields available):
${JSON.stringify(metaSchema.tables.map(t => ({
  name: t.displayName,
  fields: t.fields.filter(f => !f.isHidden).map(f => ({
    name: f.displayName, type: f.type
  }))
})), null, 2)}

USER PERMISSIONS: ${ctx.role === 'SUPER_ADMIN' ? 'Full access' :
summarisePermissions(ctx.permissions)}

You can help this user:
- Query data (you will be given tool access to read data)
- Configure the system (roles, workflows, fee structures)
- Generate reports
- Answer questions about how to use the platform
- Identify anomalies in school data

Always be concise. Prefer action over explanation unless the user asks for help.
  `.trim();
```

```
}
```

8.2 Agent Tool Definitions

The agent is given a set of tools it can call against the school's data plane. All tool calls are permission-checked against the requesting user's role before execution.

Tool	Description & Permissions
query_school_data	Execute a read-only SQL query against the school database. Params: table (string), filters (object), fields (string[]). Returns: rows array. Permission: READ on target table.
create_record	Insert a new record. Params: table, data object. Returns: created record. Permission: CREATE on target table.
update_record	Update an existing record. Params: table, id, data. Permission: UPDATE on target table.
apply_schema_change	Send a SchemaChangeDescriptor to the SME. Only available to users with CANVAS_EDIT permission. Returns: success/error.
send_notification	Send a push/SMS/WhatsApp notification to a user or group. Params: recipients, message, channel.
generate_report	Queue a background report generation job. Returns: jobId for polling.
get_screen_data	Fetch the current module's data for display in the agent's response (so it can reference it accurately).

8.3 Voice Pipeline

```
// Voice agent flow:
//
// 1. User speaks → MediaRecorder API captures audio blob
// 2. Audio blob → POST /api/v1/ai/voice/transcribe
// 3. Server → Sarvam AI STT API (Indian English / Hindi)
//    If confidence < 0.7 → fallback to Whisper
// 4. Transcript → AI agent chat pipeline (same as text)
// 5. Agent response text → POST to Sarvam AI TTS API
// 6. Audio response stream → returned to browser
// 7. Browser plays audio via Web Audio API

async function transcribeAudio(audioBlob: Buffer, language: string):
Promise<string> {
  const sarvamResult = await sarvamClient.transcribe({
    audio:    audioBlob,
    language: language,   // 'en-IN', 'hi-IN', 'te-IN', etc.
    model:    'saarika:v2',
  });
  if (sarvamResult.confidence >= 0.7) return sarvamResult.transcript;
  // Fallback to Whisper
  return await whisperClient.transcribe({ audio: audioBlob, language });
}
```

9. Multi-Language System

The platform supports English, Hindi, Telugu, Tamil, Kannada, and Marathi from v1.0. The i18n system must handle: static UI strings, dynamically generated field names (created via the canvas), date/number formatting, and right-to-left considerations (not needed for these six languages, but architecture must not block future Arabic/Urdu support).

9.1 Architecture

Layer	Implementation
Static strings	next-intl library with JSON locale files per language. Key-based: t('students.addStudent') → 'Add Student' / 'छात्र जोड़ें'. Files stored in /locales/{lang}.json. Never hardcode UI strings.
Dynamic field names	Canvas-created fields have a displayName per language stored in the meta-schema. MetaField.displayNames: Record<LangCode, string>. When field is created, other language names default to the English name until translated.
Date formatting	Intl.DateTimeFormat with locale. Indian date formats: DD/MM/YYYY. Calendar: Gregorian (schools use Gregorian, not Vikram Samvat).
Number formatting	Intl.NumberFormat with locale 'en-IN' for Indian number system (lakhs, crores). Currency: INR with ₹ symbol.
Font loading	Google Fonts Noto Sans covers all 6 scripts. Single font family, multiple unicode ranges. No flash of unstyled text — fonts preloaded in Next.js.
Translation workflow	Admin of any school can edit display names for any field in any language via the Settings module. Super admin sees all language variants in a translation matrix.

10. Mobile App Architecture — React Native 0.83

The mobile app targets parent and student portals. It uses React Native 0.83 (New Architecture / Fabric renderer) for native performance. The app shares TypeScript types, API client code, and business logic with the web app via the Turborepo monorepo.

10.1 Monorepo Package Structure

```
// Turborepo monorepo layout
apps/
  web/           — Next.js 16 web application
  mobile/        — React Native 0.83 iOS + Android app
  api/           — Fastify v5 API server
packages/
  types/         — Shared TypeScript interfaces (MetaSchema, PermissionSet,
etc.)
  api-client/    — Typed fetch client shared by web and mobile
  ui-tokens/     — Design tokens (colors, spacing, typography) — web uses CSS
vars,
                   mobile uses StyleSheet.create from same source
  schema-engine/ — SME client library (sends descriptors to API)
  i18n/          — Shared translation keys and locale files
  utils/         — Date formatting, currency, validation — shared
```

10.2 Mobile-Specific Architecture

Feature	Implementation
Navigation	React Navigation v7 with native stack. Bottom tab navigator for main sections (Home, Attendance, Fees, Results, Notifications). Deep linking for notification tap-through.
State	Zustand (same as web). Shared store definitions from packages/types. Mobile-specific slices for offline queue and biometric state.
Offline queue	For attendance marking and fee acknowledgment: actions queued in AsyncStorage if offline. Synced to server on reconnect via a background BullMQ-compatible queue. Conflict resolution: last-write-wins with server timestamp.
Push notifications	Firebase Cloud Messaging (FCM) for Android, APNs for iOS. Notification payload includes deep link URL. Rich notifications for fee reminders show amount due + pay now button.
Biometric auth	react-native-biometrics. Face ID on supported iOS, fingerprint on Android. Biometric unlocks a stored encrypted refresh token. If biometric fails 3x, falls back to PIN.
Image handling	react-native-fast-image for performant cached image loading (profile photos, document thumbnails). All images served from signed GCS URLs via our API proxy.

11. Monitoring, Observability & Alerting

System	Implementation
Error Tracking	Sentry — web, mobile, and API. Every unhandled exception captured with full stack trace, tenant context (schoolId, userId), and breadcrumbs. PII scrubbing: student names, fees, salaries stripped from Sentry payloads.
API Metrics	Fastify Prometheus plugin exports request rates, latency percentiles (p50, p95, p99), error rates per route. Grafana dashboards per school tier.
Database Monitoring	GCP Cloud SQL built-in monitoring: CPU, memory, storage, connections, slow query log. Alert: any query >500ms in production.
Schema Consistency	Nightly cron job: compare actual pg_catalog.pg_columns against _meta_schema for every active school. Discrepancies logged and PagerDuty alert fired.
Provisioning Monitor	Any school stuck in a non-terminal provision state for >30 minutes fires a PagerDuty alert. Our ops dashboard shows all schools with current provision state.
Billing Monitor	Daily check: any school with status=ACTIVE but subscription.status=past_due fires a Slack alert. Grace period: 7 days before suspension.
Cost Anomaly Detection	GCP Budget Alerts: if any school's Cloud SQL instance cost exceeds 2x their tier's expected cost in a billing period, alert ops team to investigate (possible data explosion or unintended large table).

Document Status & Next Steps

This document covers the complete technical architecture: system topology, Control Plane schema, Provisioning Pipeline (with state machine and failure recovery), Schema Migration Engine (meta-schema registry, change descriptors, SQL generation, rollback algorithm, all edge cases), Canvas Builder (Zustand state model, diff algorithm, permission system), API layer design, Authentication & RBAC, AI Agent context assembly and tool design, Multi-language system, Mobile architecture, and Monitoring.

Next engineering documents to produce: (1) Database migration versioning runbook — how the team manages core schema changes across all active schools. (2) CI/CD pipeline specification — GitHub Actions workflows for build, test, and deploy. (3) Security audit checklist — pre-launch penetration testing scope. (4) API contract document — OpenAPI spec for all endpoints.